Self-Healing RAG Platform

Product Name

Self-Healing RAG: Intelligent Retrieval-Augmented Generation System with Automatic Error Recovery

Vision

Build an enterprise-grade RAG platform capable of detecting retrieval failures, hallucinations, and missing context, then autonomously correcting itself before delivering responses.

Unlike traditional RAG systems that generate answers from potentially irrelevant documents, the platform continuously evaluates retrieval quality and answer grounding, triggering corrective actions whenever confidence drops below defined thresholds.

Problem Statement

Traditional RAG systems suffer from:

1. Poor retrieval quality
2. Hallucinated responses
3. Missing context
4. Ambiguous user queries
5. Low answer reliability

Current Workflow:

User → Retrieval → LLM → Response

If retrieval fails, the entire response becomes unreliable.

Goal:

Create a self-healing architecture capable of:

- Detecting retrieval failures
- Rewriting poor queries
- Re-running retrieval
- Verifying generated responses
- Providing confidence scores
- Ensuring answer grounding

Objectives

Primary Objectives

- Increase retrieval relevance
- Reduce hallucinations
- Improve answer confidence
- Improve user trust

Secondary Objectives

- Support enterprise document search
- Support PDF knowledge bases
- Enable source citation
- Provide evaluation dashboards

Target Users

Primary

- Enterprises
- Research teams
- Universities

Secondary

- Customer support systems
- Internal knowledge assistants
- Document intelligence platforms

Functional Requirements

Module 1: Document Ingestion

Input Formats:

- PDF
- DOCX
- TXT
- Markdown

Process:

Document Upload ↓ Chunking ↓ Embedding Generation ↓ Pinecone Storage

Features:

- Automatic chunking
- Metadata extraction
- Duplicate detection
- Incremental updates

Module 2: Embedding Pipeline

Embedding Model:

BAAI BGE-M3

Output:

1024-dimensional embeddings

Storage:

Pinecone Index

Metadata:

{ "document_name":"", "page_number":"", "chunk_id":"", "source":"" }

Module 3: Query Analyzer

Responsibilities:

- Detect ambiguity
- Detect incomplete questions
- Detect spelling issues

Example:

User: "attention in transformers"

Rewritten:

"Explain self-attention mechanism used in transformer architecture."

Module 4: Multi Query Generator

Generate 3–5 semantic variations.

Example:

Original: "What is RAG?"

Generated:

- Retrieval Augmented Generation
- RAG architecture
- RAG workflow
- How RAG works
- Components of RAG

All queries perform retrieval.

Results merged.

Module 5: Retrieval Engine

Database:

Pinecone

Search Type:

Hybrid Search

Components:

- Dense Search
- Sparse Search
- Metadata Filters

Top K:

20

Module 6: Reranking Engine

Purpose:

Improve relevance.

Input:

Top 20 Chunks

Model:

BGE Reranker

Output:

Top 5 Chunks

Module 7: Context Evaluator

Purpose:

Determine whether retrieved context is sufficient.

Output:

{ "context_score":0.84, "needs_retry":false }

If score < 0.70

Trigger Self-Healing Pipeline

Module 8: Self-Healing Engine

Healing Actions:

1. Query Rewrite
2. Retrieve Again
3. Expand Search Scope
4. Increase Top K
5. Use Metadata Filters
6. Semantic Expansion

Maximum retries:

3

Module 9: Answer Generator

Model:

Llama 3 8B via Ollama

Prompt Template:

- Answer only from context
- Cite sources

- State uncertainty if context missing

Output:

Response Sources Confidence Score

Module 10: Hallucination Detector

Purpose:

Verify generated answer against retrieved context.

Model:

Llama 3 Verifier Agent

Checks:

- Unsupported claims
- Missing citations
- Contradictions

Output:

{ "grounded":true, "score":0.91 }

If grounded=false

Regenerate answer.

Module 11: Citation Engine

Every response must contain:

- Document name
- Page number
- Chunk reference

Example:

"Transformers use self-attention [Document A, Page 14]."

Module 12: Confidence Scoring

Factors:

- Retrieval relevance
- Context completeness
- Hallucination score
- Citation coverage

Output:

Confidence: 92%

Non Functional Requirements

Latency:

< 5 seconds

Concurrent Users:

100+

Availability:

99%

Scalability:

Horizontal scaling

Security:

- JWT Authentication
- HTTPS
- API Keys

System Architecture

Frontend (React) ↓ FastAPI Backend ↓ Query Analyzer ↓ Multi Query Generator ↓ Pinecone Retrieval ↓ BGE Reranker ↓ Context Evaluator ↓ Self-Healing Layer ↓ Llama 3 Generator ↓ Hallucination Verifier ↓ Response

---

Tech Stack

Frontend

- React
- Tailwind CSS

Backend

- FastAPI
- Python

Vector Database

- Pinecone

LLM

- Ollama
- Llama 3 8B

Embeddings

- BGE-M3

Reranker

- BGE-Reranker-Large

Orchestration

- LangGraph

Evaluation

- RAGAS
- LangSmith

Deployment

- Docker
- Nginx
- AWS EC2

Monitoring

- Prometheus
- Grafana

Evaluation Metrics

Retrieval

- Recall@K
- Precision@K
- MRR

Generation

- Faithfulness
- Context Precision
- Context Recall

Business

- User Satisfaction
- Average Confidence
- Retry Success Rate

Future Enhancements

Phase 2

- Agentic RAG
- Web Search Fallback
- Graph RAG
- Knowledge Graph Integration

Phase 3

- Multimodal RAG
- Image Understanding
- Audio Understanding
- Video Search

Success Criteria

- 90%+ grounded responses
- 85%+ retrieval relevance
- Hallucination rate below 5%
- Average confidence above 85%
- Response time below 5 seconds